Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3631295.3631396

RESEARCH-ARTICLE

GreenCourier: Carbon-Aware Scheduling for Serverless Functions

MOHAK CHADHA, Technical University of Munich, Munich, Bayern, Germany

THANDAYUTHAPANI SUBRAMANIAN, Technical University of Munich, Munich, Bayern, Germany

EISHI ARIMA, Technical University of Munich, Munich, Bayern, Germany

MICHAEL GERNDT, Technical University of Munich, Munich, Bayern, Germany

MARTIN SCHULZ, Technical University of Munich, Munich, Bayern, Germany

OSAMA ABBOUD, Huawei Technologies Deutschland GmbH, Dusseldorf, Nordrhein-Westfalen, Germany

Open Access Support provided by:

Technical University of Munich

Huawei Technologies Deutschland GmbH

PDF Download
3631295.3631396.pdf
02 March 2026
Total Citations: 24
Total Downloads: 413

Published: 11 December 2023

Citation in BibTeX format

Middleware '23: 24th International Middleware Conference
December 11 - 15, 2023
Bologna, Italy

Conference Sponsors:
ACM

WoSC '23: Proceedings of the 9th International Workshop on Serverless Computing (December 2023)
https://doi.org/10.1145/3631295.3631396
ISBN: 9798400704550

GreenCourier: Carbon-Aware Scheduling for Serverless Functions

Mohak Chadha[1], Thandayuthapani Subramanian[1], Eishi Arima[1], Michael Gerndt[1], Martin Schulz[1], Osama Abboud[2]
[1]{firstname.lastname, martin.w.j.schulz}@tum.de, Technische Universität München, Germany
[2]{firstname.lastname}@huawei.com, Huawei Technologies, Germany

ABSTRACT

This paper presents GreenCourier, a novel scheduling framework that enables the runtime scheduling of serverless functions across geographically distributed regions based on their carbon efficiencies. Our framework incorporates an intelligent scheduling strategy for Kubernetes and supports Knative as the serverless platform. To obtain real-time carbon information for different geographical regions, our framework supports multiple marginal carbon emissions sources such as WattTime and the Carbon-aware SDK. We comprehensively evaluate the performance of our framework using the Google Kubernetes Engine and production serverless function traces for scheduling functions across Spain, France, Belgium, and the Netherlands. Results from our experiments show that compared to other approaches, GreenCourier reduces carbon emissions per function invocation by an average of 13.25%.

CCS CONCEPTS

• **Computer systems organization → Cloud computing**; • **Social and professional topics → Sustainability**.

KEYWORDS

Serverless Computing, Function-as-a-Service, Sustainable Serverless Computing, Carbon Efficiency

ACM Reference Format:
Mohak Chadha, Thandayuthapani Subramanian, Eishi Arima, Michael Gerndt, Martin Schulz, Osama Abboud. 2023. GreenCourier: Carbon-Aware Scheduling for Serverless Functions. In *9th International Workshop on Serverless Computing (WoSC '23), December 11–15, 2023, Bologna, Italy*. ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3631295.3631396

1 INTRODUCTION

The combustion of fossil fuels for the production of electricity is a prominent driver of global greenhouse gas emissions, which has far-reaching implications for climate change on a planetary scale [4]. According to a recent report [39] from the U.N. Intergovernmental Panel on Climate Change, the world is rapidly approaching the Paris agreement goal [9] of limiting the Earth's average temperature increase to 1.5°C by the end of the 21st century. Exceeding this limit for a sustained duration can lead to irreversible damage to the environment and an increased probability of climate catastrophes [19]. A significant amount of the world's total electricity is consumed by datacenters today. For instance, datacenters worldwide consumed 205 TWh of electricity in 2018 [34], exceeding the annual electricity consumption of countries such as Ireland and Luxembourg [37]. With the increasing migration rate of software to the cloud and the additional demand from novel domains such as Internet of Things (IoT) [24, 42], these datacenters are forecasted to consume between 3% to 13% of the world's total electricity by 2030 [23].

To offset the carbon footprint from operating these datacenters, most technology companies have invested in the generation of renewable energy through sources such as wind and solar [14, 15]. For instance, Google has publicly announced its intention to achieve carbon neutrality by 2030 [15]. For every one MWh of renewable energy generated, these companies are awarded renewable energy credits (RECs) [21]. Following this, the issued RECs are used by these companies to claim *net-zero* carbon emissions annually. However, due to the fluctuating nature of renewable energy [41],i.e., unreliability and non-availability in unlimited amounts at a single geographical location at all times, these datacenters continue to consume carbon-intensive energy repeatedly during the day when the energy supply from carbon-free sources is insufficient. According to a recent study [35], 50% of the energy used in datacenters is by idle resources. To this end, an emerging cloud computing paradigm called "Serverless Computing" has the potential to reduce wasted energy consumption from resource idling and resource underutilization in datacenters.

Function-as-a-Service (FaaS) is the computational concept of serverless computing. In FaaS, developers implement fine-grained pieces of code called *functions* that are packaged independently in containers and uploaded to a FaaS platform. Several open-source and commercial FaaS platforms such as Knative [33] and Google Cloud Functions (GCF) [20] are currently available. FaaS enables developers to focus only on the application logic while all responsibilities around resource provisioning, scaling, and related management tasks are automatically handled by the cloud provider. In addition, the FaaS computing model offers several advantages such as rapid scalability during request bursts, automatic scaling to zero when resources are unused, and an attractive pricing and development model [24].

Serverless functions are *ephemeral*, i.e., short-lived, and *event-driven*, i.e., these functions only get executed in response to external triggers such as HTTP requests. Moreover, these functions are *stateless*, i.e., any application state needs to be persisted in external storage or databases. For commercial FaaS providers, the functions are executed in the cloud providers' traditional IaaS virtual machine (VM) offerings hosted in their internal geographically distributed datacenters [25, 31]. Typically, these cloud providers receive around

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WoSC '23, December 11–15, 2023, Bologna, Italy
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0455-0/23/12…$15.00
https://doi.org/10.1145/3631295.3631396

one billion function invocations each day [38]. For all commercial FaaS platforms, the geographical regions for function execution are pre-selected by the users during function deployment. As a result, the varying *carbon intensities* of the different geographical regions are not considered for function execution by any commercial FaaS platform. The carbon intensity of a geographical region describes the amount of carbon emitted per kWh of energy produced. The intensity can vary significantly across regions and even within the same region over time [41]. Towards sustainable serverless computing, our key contributions are:

- We implement and present `GreenCourier`, a novel scheduling framework that intelligently schedules serverless functions across geographically distributed Kubernetes clusters to minimize carbon emissions for function execution at runtime. Our framework is available at: https://github.com/kky-fury/carbon-sched.
- We comprehensively evaluate our framework using the Google Cloud platform and production function traces [38] on different serverless functions against other scheduling strategies wrt generated carbon emissions and function response times.
- We quantify and analyze the overhead for function scheduling in our framework.

2 GREENCOURIER

2.1 Overview

The purpose of `GreenCourier` is to dynamically schedule serverless functions across different geographical regions depending on their carbon efficiency. The different system components of our framework are shown in Figure 1. GreenCourier builds on Kubernetes (K8s) which is the *de-facto* platform for container orchestration in the cloud. As the FaaS platform, our framework utilizes the serving component of Knative. We chose Knative since it is utilized in production by Google in their FaaS and serverless Container-as-a-Service offerings [20, 36]. For establishing geographically distributed multi-cluster K8s topologies, our framework uses Liqo [18]. In multi-cluster K8s environments, independent clusters can be located in different datacenters across different countries leading to several challenges such as synchronization of cluster objects for pod [1] scheduling and communication between services in different clusters. Liqo is an open-source project that provides a multi-cluster control plane that enables (i) the seamless scheduling of pods across different clusters and (ii) the automatic dynamic discovery of new clusters as they are added to the environment. In addition, it provides a network fabric [8], which extends the K8s networking model across all clusters in the environment, enabling the inter-communication between services. For connecting two individual clusters, Liqo uses the *peering* methodology [10]. Peering involves establishing a unidirectional resource and service consumption relationship between two clusters, where one cluster, i.e., the consumer is granted the ability to offload pods to another remote cluster, i.e., the provider, but not vice-versa. In this scenario, the consumer creates an outgoing peering connection towards the provider, while the provider is subjected to an incoming peering connection from the consumer. This allows for the two clusters to interconnect and enables the consumer to access the services

[1]We use the term pod and function interchangeably in this paper since they are the same from K8s's perspective.

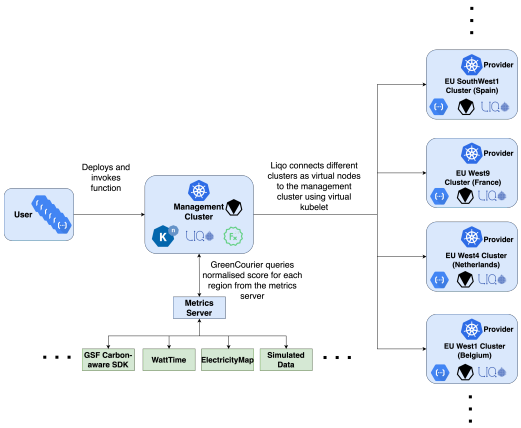


Figure 1: `GreenCourier` System Architecture.

and resources provided by the provider cluster. In our framework, the different geographically distributed clusters are the providers, while the management cluster is the consumer. For connecting the independent provider clusters to the management cluster, Liqo uses Virtual Kubelet (VK) [16]. VK cloaks a provider cluster as a virtual node and connects it to the management cluster. For carbon-aware scheduling decisions at runtime, our framework obtains normalized carbon score data from the metrics server (§2.2). Following this, the function is scheduled to the region with the highest current carbon score. The management cluster is the central component of our framework, where the `GreenCourier` scheduler (§2.4), Knative, Liqo, VK, and the metrics server are deployed. From the perspective of the user, using our framework is extremely uncomplicated. A user simply deploys their function written in any programming language supported by Knative on our management cluster. Following this, the function can be invoked using the returned `url` after function deployment.

2.2 Metrics Server

The metrics server is responsible for calculating and normalizing the carbon-efficiency scores for the different geographical regions. It exposes a REST API that can be efficiently utilized by the `GreenCourier` scheduler (§2.1) to obtain the normalized carbon intensity scores. To obtain data about carbon emissions, our metrics server currently supports two sources, i.e., WattTime (default) [17] and the Carbon-aware SDK [1]. WattTime leverages real-time grid data and advanced algorithmic and forecasting techniques to present unprecedented visibility into the marginal operating emission rate (MOER) of the local electricity grid. MOER is a quantification of the rate of carbon emissions produced by electricity generators in response to variations in the demand for energy on the local electrical grid, at a specific point in time and geographical location. WattTime provides energy data in units of pounds of emissions per megawatt-hour, i.e., lbsCO2/MWh. The Carbon-aware SDK was developed by the Green Software Foundation (GSF) to enable the development of advanced software solutions that embody a carbon-conscious approach to energy consumption. Towards this, it provides a REST API and command line interface that can be utilized by carbon-aware applications to obtain aggregated MOER data from different third-party sources such as WattTime. In addition, it provides a standardized interface for obtaining MOER data in units of grams of emissions per kilowatt-hour, i.e., gCO2/kWh.

Algorithm 1: Carbon-aware scheduling strategy.

1 **Scoring Regions:**
2 **Function** `Calculate_Scores`(*PodObject, ListofNodes*):
3 **for** *each Node in ListofNodes* **do**
4 *Region* = Node.Annotation("region").
5 *NodeScore* = Retrieve carbon score from metrics server for *Region*.
6 Update and store *NodeScore*.
7 **end**
8 Normalise node scores.
9 *Node* = Get the node with the highest score.
10 Assign *PodObject* on *Node*.
11 **return** *PodObject*

For normalizing the current carbon scores, we use the *min-max* normalization technique in our metrics server. Furthermore, our metrics server can be easily extended to support other carbon data sources such as Electricity Maps [3] and simulated data [41].

2.3 Carbon-aware scheduling of serverless functions

The carbon-aware scheduling strategy in `GreenCourier` is implemented using the Scheduling framework [11] provided by K8s. The scheduling framework defines various extension points exposed by the K8s scheduler API, where different plugins can be enabled and executed during pod scheduling. Scheduling of pods in K8s occurs in two phases, i.e., the scheduling cycle and the binding cycle. The former involves selecting a node for executing the pod, while the latter involves applying that decision to the cluster. Node selection in K8s involves two consecutive stages, i.e., filtering and scoring. The filtering phase involves the execution of a set of predicate plugins that define strict limitations on a node to be eligible for executing a pod. For instance, the *NodeResourcesFit* filtering plugin checks whether the resources requested by a pod are available on a particular node. Our scheduling strategy supports multiple predicate plugins provided by K8s [7] such as *NodeResourcesFit*, *TaintToleration*, and *NodeAffinity*. In contrast, the scoring phase evaluates a node based on a series of soft constraints called priorities. These priorities may either be specified in the pod definition or could stem from broader constraints associated with the nodes or the cluster. In this phase, every node is evaluated by the enabled priority plugins, which assign every node a score. For instance, the *ImageLocality* priority plugin, assigns a node with a high score if the container image requested by the pod is locally present on that node. On completion of the scoring phase, the node with the highest aggregated score is selected for pod assignment. To enable the carbon-aware scheduling of serverless functions across geographically distributed clusters, we implement a custom scoring plugin for K8s. Algorithm 1 describes our custom scoring strategy. *ListofNodes* represent the eligible nodes available for pod assignment after the filtering phase, while *PodObject* represents the current pod that needs to be scheduled. Initially, our scoring plugin iterates over the eligible set of nodes and obtains the current carbon score associated with the region where the node is located, from the metrics server (Lines 3-5). The region of the node is determined by reading the annotations set by the administrator during cluster creation (Line 4). We store and update the obtained carbon scores in a key-value-based data structure (Line 5). To reduce overhead for scheduling, we cache the obtained carbon scores for a particular region for five minutes locally. We chose this granularity since both WattTime and Carbon-aware SDK provide updated data in five-minute intervals (§2.2). After iterating across all nodes, we normalize the obtained node scores between zero to 100 (Line 8). Following this, we select the node with the highest score (Line 9) and assign the pod specification with its name (Line 10).

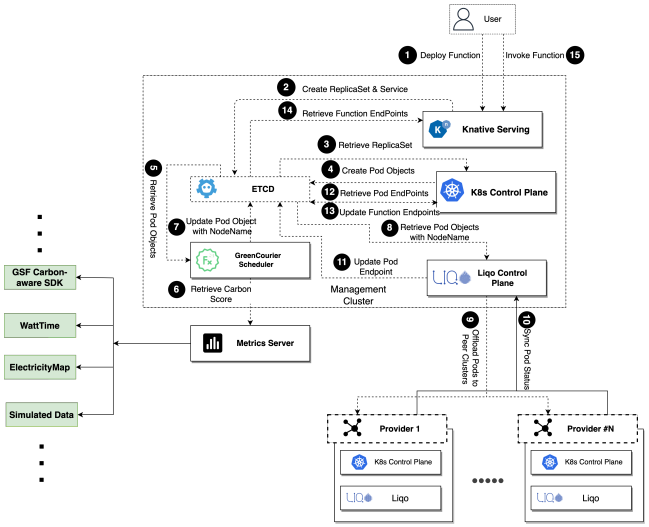


Figure 2: Workflow for serverless function scheduling in `GreenCourier`.

2.4 Putting It All Together

Figure 2 shows the detailed workflow for scheduling serverless functions in `GreenCourier`. First, the user defines the serverless function as a `YAML` specification file that contains information regarding its container image and runtime configurations (❶). To use our carbon-aware scheduler, the user must set the field *schedulerName* to `kube-green-courier` in the specification file. Using this specification file, the user sends a request to the Knative serving control plane to create the function. Following this, the associated K8s objects are created by Knative (❷). This leads to a state change in `etcd`, which is detected and retrieved by K8s (❸). After this, K8s creates the associated Pod objects for the function and updates `etcd` (❹). Our custom scheduler listens for any changes to the state associated with Pod objects for Knative functions. As a result, the newly created pod objects are retrieved by our scheduler (❺). Following this, our implemented priority plugin (§2.3) is executed, which involves obtaining the carbon-efficiency scores for the different geographically distributed nodes (❻). On selection of the optimal node for pod assignment, our scheduler modifies the `nodeName` field in the pod specification and pushes the updated specification to `etcd` (❼). Liqo [18] acts as an alternative to `kubelet` [6] in our scheduling framework. It listens for pod objects that are assigned to the different virtual nodes connected to our management cluster and retrieves them from `etcd` (❽). After this, Liqo initiates the pod offloading process (❾) to the chosen provider cluster (§2.1). On reconciliation of the pod creation request, Liqo updates the

pod's status in the management cluster (10). Following this, Liqo updates the local routing tables of the management cluster to ensure that all network traffic to the offloaded pod is sent through the network fabric established after the peering process (§2.1). For this, it updates the endpoint, i.e., IP of the pod associated with the serverless function (11). The updated pod endpoints are retrieved by K8s (12) which triggers an update for the deployed Knative function (13). Following this, Knative updates the created pod objects of the function with the appropriate IP of the pod running in the provider cluster (14). Finally, the function can be invoked by the user through the `url` received after function deployment (15). On multiple invocations of the deployed function, the Knative pod autoscaler (KPA) increases the replica count of the deployed function to reduce function response times. In this scenario, steps 3-14 are repeated.

3 EXPERIMENTAL RESULTS

3.1 Experimental Setup

3.1.1 Experiment configuration. For analyzing the performance of our scheduling framework, we use the Google Kubernetes Engine (GKE) in standard mode [28]. We deploy our management cluster (§2.1) in the Frankfurt region, i.e., `europe-west-3a` with a configuration of 16vCPUs and 64GiB of memory. In addition, we provision four provider clusters across Madrid (`europe-southwest1-a`), Paris (`europe-west9-a`), St. Ghislain (`europe-west1-b`), and Eemshaven (`europe-west4-a`). Each provider cluster has four VMs of type `e2-standard-4`. In our setup, we only consider geographical locations within Europe due to the limitations of our WattTime license for obtaining carbon data outside it. All VMs provisioned in our setup are based on the Intel Xeon Platinum 8173M Skylake-SP processor architecture [2]. Note that by default GKE does not support the provisioning of a K8s cluster with nodes in different geographical regions.

3.1.2 Serverless functions. For our experiments, we use eight functions (§3.2,§3.3) from the `FunctionBench` [32] benchmark suite. We adapt the different functions for Knative and modify the communication protocol from HTTP to `gRPC`. All functions are implemented using the Python programming language.

3.1.3 Load Testing. For emulating actual user behavior for invoking the deployed serverless functions, we implement a custom load generator using `k6`. `k6` is an open-source performance and regression testing tool that provides a JavaScript API to develop custom load-testing scenarios. In addition, it supports both HTTP(s) and `gRPC` based protocols for communication. To ensure a representative function invocation pattern in our experiments, we use the production-level serverless function traces from Microsoft Azure functions [38]. Our load generator reads the input traces and generates different scenarios, dynamically simulating the function request pattern. To model request inter-arrival time, we use the Poisson distribution. For our experiments, we limit a load test to ten minutes and repeat it five times.

$$Software\ Carbon\ Intensity = ((E \cdot I) + M)/R \quad (1)$$

$$W.\ A.\ MOER = \frac{\sum_{i=1}^{n} (\#function\ instances\ in\ a\ region(i)\ *\ MOER(i))}{\sum_{i=1}^{n} \#function\ instances\ in\ a\ region(i)} \quad (2)$$

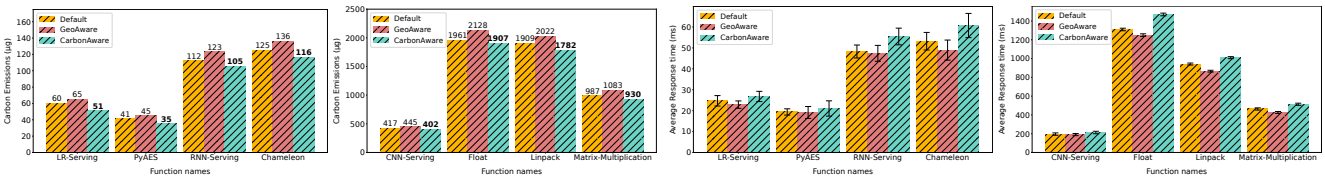
3.1.4 Metrics. To quantify and compare the performance of our scheduling framework, we use the metrics *carbon emissions*, *function response time*, *scheduling latency*, and *binding latency*. For computing the carbon emissions per function invocation, we use the Software Carbon Intensity (SCI) specification [12] provided by the GSF (§2.2). SCI provides a detailed guide for determining the carbon footprint of a software application by quantifying its associated carbon emissions. For any application, SCI can be calculated by using Equation 1. In Equation 1, `E` represents the energy consumed by the application, `I` represents the location-based marginal carbon intensity, `M` represents the embodied carbon emissions for the hardware where the application is executing, and `R` represents the functional unit such as the #requests by which an application scales. To estimate `E` for our provider clusters with 64vCPUs and 256GiB of RAM, we use the methodology described in [13]. Towards this, we use 165W as the TDP for the Intel Skylake-SP processor [5] and assume 50% CPU utilization. We choose this number since recent studies suggest that cloud computing resources are over-provisioned and not fully utilized [26]. Similar to [13], we assume that every 8GiB of RAM consumes 3W of power. In addition, 2vCPUs constitute one CPU core on GKE. Following this, we calculate E for our cluster setup as $165 \times 50\% \times 24 * 32 + 96 = 63.456$kWh. To compute `I`, we calculate the weighted average MOER as shown in Equation 2. This is because the number of function instances[2] launched after a load test can vary across regions and a weighted average constitutes a valid approximation. In Equation 2, `n` represents the total number of regions. For estimating `R`, we assume that the request traffic is uniformly distributed among all function instances. Following this, we calculate the maximum number of requests that can be served by a single function instance during a day using its response time. For instance, for a function with a response time of 200ms the `R` value would be 432000 requests per 24 hours. In our calculations, we ignore `M` since embodied carbon emissions remain unaffected by the implementation of our scheduling framework.

Function response time represents the end-to-end time to process and respond to a function invocation request. For computing the scheduling latency (§2.3), we measure the time between the *NodeAssgined* event generated by the K8s scheduler and the *PodCreation* event generated by the ReplicaSet controller. On the other hand, for computing the binding latency, we measure the time between the *NodeAssgined* event and the *PodRunning* event generated by Liqo/Kubelet.

3.2 Comparing carbon emissions

We compare the performance of our carbon-aware scheduling strategy against the default scheduling strategy in K8s and a custom `GeoAware` scheduling strategy. In contrast to our carbon-aware scheduling strategy, the `GeoAware` strategy assigns pods to nodes that are located at a closer distance to the management cluster. Similar to our carbon-aware strategy (§2.3), we implement the `GeoAware` scheduling strategy as a priority plugin in K8s. Figure 3a shows the carbon emissions in $\mu g s$ for a single function invocation across the different scheduling strategies. We quantify the amount of carbon emitted using the SCI specification as described in §3.1.4. For the different serverless functions, using `GreenCourier` leads to

[2] A single function instance is equivalent to a single pod in K8s.



(a) Comparing carbon emissions. **(b) Comparing response times.**

Figure 3: Comparing carbon emissions and response times for the different serverless functions (§3.1.2) and scheduling strategies.

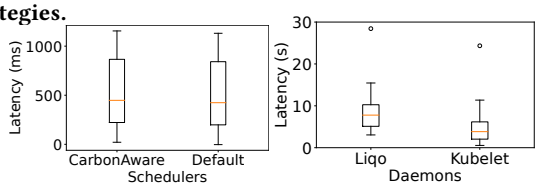


Figure 4: Comparing scheduling latency (left) and binding latency (right). in our scheduling framework.

8.7% and 17.8% reduction in carbon emissions on average as compared to the default and the `GeoAware` scheduling strategies respectively. In our experimental setup (§3.1), the most carbon-efficient region is `europe-southwest1-a`, followed by `europe-west9-a`, `europe-west1-b`, and `europe-west4-a`. On the other hand, the region closest to the management cluster is `europe-west1-b`, followed by `europe-west4-a`, `europe-west9-a`, and `europe-southwest1-a`. As a result, the `GeoAware` strategy places serverless functions in regions with low carbon efficiencies leading to a higher carbon footprint. The default scheduling strategy performs better than the `GeoAware` strategy since it relies on the `PodTopologySpread` K8s plugin that tries to evenly spread functions across all provider clusters to maximize availability.

While the geo-based scores for the different provider clusters always remain constant, the carbon-scores for the different regions change every five minutes (§2.3). However, in our experiments, we observed that `europe-southwest1-a` and `europe-west9-a` were always the most carbon-efficient regions. It is important to note that the carbon efficiencies of the different geographical regions selected for deploying the provider clusters influence the carbon reductions achieved with our scheduling strategy (§2.3). With significant differences between the carbon scores of the different geographical regions, the reductions in carbon emissions will be significantly higher as compared to the other scheduling strategies.

3.3 Comparing response times

Figure 3b shows the average response times for the different serverless functions and scheduling strategies. From our experiments, we observe that the `GeoAware` scheduling strategy leads to the least average response times for the different serverless functions, followed by the default and the carbon-aware scheduling strategies. Our carbon-aware scheduling strategy leads to geometric mean (GM) average slowdown of 10.26% and 16.24% across all functions as compared to the default and `GeoAware` scheduling strategies respectively. As compared to the default scheduling strategy, the `GeoAware` strategy leads to a GM average speedup of 4.2% across all functions. The performance degradation with our carbon-aware scheduling strategy can be attributed to the selection of carbon-efficient regions, i.e., `europe-southwest1-a` and `europe-west9-a` for serverless function execution. These geographical regions are furthest away from our management cluster, i.e., (`europe-west3-a`) leading to higher average function response times. On the other hand, the `GeoAware` strategy maximizes the execution of functions in regions closest to our management cluster leading to significantly less average response times. Note that the cost for all three strategies is equal since the users are only billed for the function execution time in the FaaS computing model.

3.4 Analyzing overhead

Figure 4 shows the comparison between the scheduling latencies for our carbon-aware scheduler and the default scheduler in K8s (§2.3,§3.1.4). The yellow line represents the median observed latency, while the whiskers represent the minimum and maximum latency values. We omit the comparison with the `GeoAware` scheduling strategy since its implementation is similar to our carbon-aware strategy. From our experiments, we observe that the `GreenCourier` scheduler takes 539ms on average for scoring/assigning carbon-efficient nodes to pods as compared to 515ms taken by the default K8s scheduler.

To evaluate the binding latency of our scheduling framework, we compare it against a traditional K8s setup that uses a single cluster with multiple worker nodes in the same geographical region (§3.1.1). In the traditional setup, kubelet is responsible for executing pods on worker nodes and managing their state in the K8s control plane (§2.4). However, in our scheduling framework which uses Liqo for creating a multi-cluster K8s topology, virtual kubelet (VK) (§2.1) is used to impersonate an independent cluster as a node from the perspective of the management cluster. Figure 4 shows the binding latencies for the two setups. With `GreenCourier` and Liqo, we observe an average pod binding latency of 8.28s as compared to 4.53s with kubelet. The increased latency in our framework can be attributed to two reasons. First, due to the resource abstraction added by Liqo via VK, an additional layer of synchronization is required in `GreenCourier` for communicating state information between the management cluster and the provider clusters. Second, frequent communication across geographically distributed clusters via the public internet increases the binding latency. In contrast, all communication in the traditional setup is done within a virtual private cloud (VPC).

4 RELATED WORK

Scheduling serverless functions. While developing strategies for scheduling serverless functions is an active research area [29, 30, 40], no prior work has explored the carbon-aware scheduling of serverless functions across geographically distributed regions. Kaffes et al. [30] propose a centralized scheduler that can schedule functions at a processor core-level granularity, to better address the unique characteristics of serverless workloads. They demonstrate

with extensive experiments that their scheduler provides enhanced elasticity to users for function execution and is adaptive to varying function request demands. Suresh et al. [40] propose FnSched, a scheduling strategy that aims to minimize resource cost for cloud service providers for executing serverless functions. They implement their strategy on OpenWhisk and demonstrate upto 55% reduction in host utilization as compared to other strategies, albeit reducing function execution performance. Jindal et. al. [29] propose Courier, a framework to load balance scheduling of serverless functions across heterogeneous FaaS deployments such as GCF [20], AWS Lambda, and OpenWhisk. Courier incorporates an adaptive weighted round-robin scheduling strategy that dynamically adapts the weights of the different FaaS platforms depending on current function performance.

Sustainable scheduling for datacenters. Most prior work on sustainable scheduling for datacenters has focused on developing energy-aware strategies [22, 43], with only a few approaches that target maximizing the use of green energy [27]. However, no prior work in the literature enables the runtime scheduling of serverless functions across geographically distributed datacenters to reduce carbon emissions. To reduce energy consumption for idle resources, Alahmadi et al. [22] propose a novel strategy for scheduling, sharing, and migration of VMs in datacenters. For real-time energy-aware scheduling of tasks, Zhu et al. [43] propose an energy-aware rolling horizon (EARH) algorithm for scheduling tasks in batches on VMs.

5 CONCLUSION AND FUTURE WORK

In this paper, we took the first step towards sustainable serverless computing and presented GreenCourier, a scheduling framework that enables runtime carbon-aware scheduling of serverless functions across geographically distributed multi-Kubernetes clusters. With comprehensive experiments using production function traces, we demonstrated that our carbon-aware scheduling strategy leads to 13.25% average reductions in carbon emissions per function invocation as compared to two other strategies. In the future, we plan to extend the management control plane of our framework to make it highly available and disaggregated. Furthermore, we plan to adapt our scheduling strategy to account for service level objectives while scheduling serverless functions.

6 ACKNOWLEDGEMENT

The research leading to these results was funded by the German Federal Ministry of Education and Research (BMBF) in the scope of the Software Campus program under the grant agreement 01IS17049. Google Cloud credits in this work were provided by the *Google Cloud Research Credits* program with the award number 64c92de5-fb62-4386-8c5b-ff3f480390bb.

REFERENCES

[1] [n.d.]. Carbon Aware SDK by Green Software Foundation. https://github.com/Green-Software-Foundation/carbon-aware-sdk.
[2] [n.d.]. CPU platforms. https://tinyurl.com/2p9vp9da.
[3] [n.d.]. electricityMap. https://electricitymap.org/.
[4] [n.d.]. Energy and Climate. https://tinyurl.com/2p8khujf.
[5] [n.d.]. Intel Xeon 8173M. https://tinyurl.com/2zxckpwc.
[6] [n.d.]. Kubelet. https://tinyurl.com/2p98rtu4. Accessed: 2022-06-01.
[7] [n.d.]. Kubernetes Scheduler. http://tiny.cc/1tp4vz.
[8] [n.d.]. Network Fabric - Liqo. https://tinyurl.com/yd854evj.
[9] [n.d.]. Paris Agreement. https://tinyurl.com/47v3xe2r. Accessed: 2023-02-13.
[10] [n.d.]. Peering - Liqo. https://docs.liqo.io/en/v0.7.0/features/peering.html.
[11] [n.d.]. Scheduling Framework. https://tinyurl.com/4ykcw7vf.
[12] [n.d.]. SCI Specification. https://tinyurl.com/4me289md.
[13] [n.d.]. SLOs and GHGs. https://tinyurl.com/bdhdesnd.
[14] [n.d.]. Sustainability - Microsoft. https://tinyurl.com/s5bemraf.
[15] [n.d.]. Tracking Google's carbon-free energy progress. https://sustainability.google/progress/energy/. Accessed: 2022-06-01.
[16] [n.d.]. Virtual Kubelet. https://virtual-kubelet.io/.
[17] [n.d.]. WattTime. https://www.watttime.org/.
[18] [n.d.]. What is Liqo? https://docs.liqo.io/en/v0.7.0/.
[19] [n.d.]. World's Climate goal. http://tiny.cc/xsp4vz. Accessed: 2023-02-13.
[20] 2022. Cloud Functions Second Generation. https://tinyurl.com/5e6fcswh
[21] Bilge Acun et al. 2023. Carbon Explorer: A Holistic Framework for Designing Carbon Aware Datacenters. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2023)*. 118–132. https://doi.org/10.1145/3575693.3575754
[22] Abdulrahman Alahmadi et al. 2014. Enhanced First-Fit Decreasing Algorithm for Energy-Aware Job Scheduling in Cloud. In *2014 International Conference on Computational Science and Computational Intelligence*. https://doi.org/10.1109/CSCI.2014.97
[23] Anders SG Andrae and Tomas Edler. 2015. On global electricity usage of communication technology: trends to 2030. *Challenges* 6, 1 (2015).
[24] Paul Castro, Vatche Ishakian, Vinod Muthusamy, and Aleksander Slominski. 2019. The rise of serverless computing. *Commun. ACM* 62, 12 (2019), 44–54.
[25] Mohak Chadha et al. 2021. Architecture-Specific Performance Optimization of Compute-Intensive FaaS Functions. In *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. 478–483. https://doi.org/10.1109/CLOUD53861.2021.00062
[26] Eli Cortez et al. 2017. Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles* (Shanghai, China) *(SOSP '17)*. 153–167. https://doi.org/10.1145/3132747.3132772
[27] Íñigo Goiri et al. 2011. GreenSlot: Scheduling energy consumption in green datacenters. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*. 1–11. https://doi.org/10.1145/2063384.2063411
[28] Google Kubernetes Engine. 2022. . http://tiny.cc/0tp4vz
[29] Anshul Jindal et al. 2021. Courier: Delivering Serverless Functions Within Heterogeneous FaaS Deployments. In *2021 IEEE/ACM 14th International Conference on Utility and Cloud Computing (UCC'21)* (Leicester, United Kingdom) *(UCC '21)*. Association for Computing Machinery, New York, NY, USA, 10. https://doi.org/10.1145/3468737.3494097
[30] Kostis Kaffes et al. 2019. Centralized Core-Granular Scheduling for Serverless Functions. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '19)*. https://doi.org/10.1145/3357223.3362709
[31] Michael Kiener et al. 2021. Towards Demystifying Intra-Function Parallelism in Serverless Computing. In *Proceedings of the Seventh International Workshop on Serverless Computing (WoSC7) 2021* (Virtual Event, Canada) *(WoSC '21)*. Association for Computing Machinery, New York, NY, USA, 42–49. https://doi.org/10.1145/3493651.3493672
[32] Jeongchul Kim and Kyungyong Lee. 2019. Functionbench: A suite of workloads for serverless cloud function service. In *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*. IEEE, 502–504.
[33] Knative. [n.d.]. https://knative.dev/docs/. Accessed 09/24/2020.
[34] Eric Masanet et al. 2020. Recalibrating global data center energy-use estimates. *Science* 367, 6481 (2020), 984–986.
[35] Panos Patros et al. 2021. Toward sustainable serverless computing. *IEEE Internet Computing* 25, 6 (2021), 42–50. https://doi.org/10.1109/MIC.2021.3093105
[36] Google Cloud Platform. 2021. https://cloud.google.com/run/.
[37] Hannah Ritchie, Max Roser, and Pablo Rosado. 2022. Energy. *Our World in Data* (2022). https://ourworldindata.org/energy.
[38] Mohammad Shahrad et al. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 205–218. https://www.usenix.org/conference/atc20/presentation/shahrad
[39] P.R. Shukla et al. 2022. Climate Change 2022: Mitigation of Climate Change. *Cambridge University Press, Cambridge, UK and New York, NY, USA*. (2022). https://doi.org/10.1017/9781009157926
[40] Amoghvarsha Suresh and Anshul Gandhi. 2019. FnSched: An Efficient Scheduler for Serverless Functions. In *Proceedings of the 5th International Workshop on Serverless Computing (WOSC '19)*. https://doi.org/10.1145/3366623.3368136
[41] Philipp Wiesner et al. 2021. Let's wait awhile: how temporal workload shifting can reduce carbon emissions in the cloud. In *Proceedings of the 22nd International Middleware Conference (Middleware '21)*. https://doi.org/10.1145/3464298.3493399
[42] Philipp Wiesner et al. 2022. Cucumber: Renewable-Aware Admission Control for Delay-Tolerant Cloud and Edge Workloads. In *European Conference on Parallel Processing*. Springer, 218–232.
[43] Xiaomin Zhu et al. 2014. Real-Time Tasks Oriented Energy-Aware Scheduling in Virtualized Clouds. *IEEE Transactions on Cloud Computing* 2, 2 (2014), 168–180. https://doi.org/10.1109/TCC.2014.2310452